

Role of the C library

When we say C library, the most commonly and widely distributed C library with a Linux based OS is glibc or GNU C library. This C library helps implement standard C functions and APIs like `print()`, `scanf()`, `malloc()`, `fopen()`, `strcpy()`, etc. These standard functions may or may not invoke system calls internally — for example, `printf()` internally invokes `write(2)` system calls. However, all these internal invocations of system calls are hidden from the user space application.

Now let's examine the role of C library when a user space application invokes a system call. For sake of simplicity let's take the example of the `open(2)` system call.

```
#include <stdlib.h>
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <errno.h>

int main ()
{
    int fd = open ("/home/shwetabh/wrapper.txt", O_CREAT|O_
RDWR);
    close(fd);
    return 0;
}
```

In the above code snippet, the user application invokes the `open(2)` system call. But as we already discussed, system calls are implemented in kernel space and cannot be directly invoked from user programs. So how does a user space application invoke `open(2)` system calls. Actually, the `open()` in the above code snippet is the wrapper function implemented in the C library (glibc). This wrapper function internally invokes the actual system call implemented in the kernel. The C library provides the wrapper for almost all the system calls implemented in Linux. Therefore, from an application program point of view, invoking a system call is similar to calling a C function.

Now we understand that glibc provides the system call wrapper and the user application invokes it, which further invokes system call. Let's now examine the major steps performed by the wrapper function for the execution of system calls. These are:

- Setting up arguments
- Calling system calls
- Checking the return values

Setting up arguments

The wrapper function performs the validation, initialisation (in some cases) and error checking of the argument provided by

the application program. In case of an error, it directly returns to the user program with the relevant 'errno'. After successful validation, wrapper functions load the system call number and argument into a specific CPU register as specified by the Linux kernel. The table below shows the detailed order in which the wrapper function or user space program should load the arguments into registers before invoking the system call.

From AMD64 architecture manual.

The Linux AMD64 kernel uses internally the same calling conventions as user-level applications. User-level applications that like to call system calls should use the functions from the C library. The interface between the C library and the Linux kernel is the same as for the user-level applications with the following differences:

1. User-level applications use as integer registers for passing the sequence `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8` and `%r9`. The kernel interface uses `%rdi`, `%rsi`, `%rdx`, `%r10`, `%r8` and `%r9`.
2. A system-call is done via the `syscall` instruction. The kernel destroys registers `%rcx` and `%r11`.
3. The number of the `syscall` has to be passed in register `%rax`.
4. System-calls are limited to six arguments, no argument is passed directly on the stack.
5. Returning from the `syscall`, register `%rax` contains the result of the system-call. A value in the range between -4095 and -1 indicates an error, it is -errno.
6. Only values of class INTEGER or class MEMORY are passed to the kernel.

Class INTEGER: This class consists of integer types that fit into one of the general-purpose registers.

Class MEMORY: This class consists of types that will be passed and returned in memory via the stack. These will mostly be strings or memory buffer. For example in `write()` system call, the first parameter is `fd` which is of class INTEGER while the second argument is the buffer which has the data to be written in the file, the class will be MEMORY over here. The third parameter which is the count - again has the class as INTEGER.

Register	Argument User Space	Argument Kernel Space
<code>%rax</code>	Not Used	System Call Number
<code>%rdi</code>	Argument 1	Argument 1
<code>%rsi</code>	Argument 2	Argument 2
<code>%rdx</code>	Argument 3	Argument 3
<code>%r10</code>	Not Used	Argument 4
<code>%r8</code>	Argument 5	Argument 5
<code>%r9</code>	Argument 6	Argument 6
<code>%rcx</code>	Argument 4	Destroyed
<code>%r11</code>	Not Used	Destroyed